Roll No: C014	Name: N Mehul
Branch: B.TECH 3RD YEAR -C.E	Batch: 2023-2027
Date of Experiment:	Date of Submission:
Grade:	Faculty: NIKITHA PANDE MAM

Aim:

- Determine **Histogram of Oriented Gradients (HOG)** for a given image
- Use different standard deviations in preprocessing to observe effect on **edge detection**
- Comment on results based on fine vs. broad edges

Algorithm for Histogram of Oriented Gradients (HOG)

1. Start

2. Input the image

3. Convert to grayscale

4. Apply Gaussian Blur with a standard deviation (σ)

```
blurred = cv2.GaussianBlur(gray, (0, 0), sigmaX=sigma)
```

5. Compute gradients using Sobel filter

```
grad_x = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
grad_y = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
```

6. Compute magnitude and angle of gradients

```
magnitude = cv2.magnitude(grad_x, grad_y)
angle = cv2.phase(grad_x, grad_y, angleInDegrees=True)
```

7. Compute Histogram of Oriented Gradients (HOG)

```
hog = cv2.HOGDescriptor()
hog_features = hog.compute(blurred)
```

8. Visualize or analyze the HOG features (optional)

cv2.imshow('Gradient Magnitude', magnitude / magnitude.max()) # Normalize for display

9. Repeat Steps 4–8 with different σ values/

10. End

CODE:

```
import cv2

import numpy as np

from google.colab.patches import cv2_imshow


image = cv2.imread("Lenna.png")


if image is None:

    print("Error: Image not found!")

    exit()


gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)


sigma = 1.0

blurred = cv2.GaussianBlur(gray, (0, 0), sigmaX=sigma)


hog = cv2.HOGDescriptor()


hog_features = hog.compute(blurred)
```

```
print("HOG Feature Vector Length:", len(hog_features))

grad_x = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
grad_y = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)

magnitude = cv2.magnitude(grad_x, grad_y)

magnitude_display = cv2.normalize(
    magnitude, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

medium_width = 300

def resize_image(img, target_width):
    h, w = img.shape[:2]
    aspect_ratio = w / h
    target_height = int(target_width / aspect_ratio)
    if len(img.shape) == 2:
        img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
    return cv2.resize(img, (target_width, target_height),
interpolation=cv2.INTER_AREA)

resized_image = resize_image(image, medium_width)
resized_gray = resize_image(gray, medium_width)
resized_magnitude = resize_image(magnitude_display, medium_width)
```

```
combined_image = np.hstack((resized_image, resized_gray,
resized_magnitude))

cv2_imshow(combined_image)
```

OUTPUT:

Conclusion:

The Histogram of Oriented Gradients (HOG) technique was used to extract important features from an image by converting it to grayscale, reducing noise with Gaussian smoothing, and computing gradient magnitude and direction. These gradients were organized into histograms, normalized, and combined to form a feature vector that represents the shape and structure of objects in the image. HOG is effective in capturing edge and texture information and is robust to changes in lighting and small shape variations. Therefore, it is widely used in applications such as object detection and image classification, making it a reliable and efficient method for image analysis.